

Iterative Game Development Approach

Thought Process

My development approach is iterative and problem-driven rather than strictly planned. Instead of designing full systems in advance, I start with a general idea of the desired outcome and gradually build and refine the solution in Unity. I usually first think through possible implementations, then begin coding and adjust the system as limitations or new ideas appear during development.

This approach allows flexibility and experimentation, especially when working on gameplay systems. Even though I do not follow a strict step-by-step plan, I always have a clear end goal and continuously guide the implementation toward it.

Inventory System

The Inventory script is a basic item management system that stores collected items and displays them through UI slots. Slots are generated dynamically in a grid layout using `CreateSlots`, while `RefreshUI` keeps the UI synchronized with the item list.

When items are added or removed, the interface updates immediately to reflect changes. The system also supports clearing the inventory and retrieving item IDs, which is useful for save/load functionality. Additionally, it allows access to the currently selected item from the UI.

GameManager System

The GameManager acts as the central system of the game, controlling UI, player state, inventory, world data, and save/load functionality. It uses a Singleton pattern to allow global access.

Input handling manages menu toggling with Escape and debug output with F1. The UI system uses a CanvasGroup to control menu visibility, interaction, and game time scale, effectively pausing and resuming gameplay. It also manages world state debugging and persistence.

Player Visual Controller

This script controls visual feedback for player movement. It rotates the player's body based on Rigidbody2D velocity and adjusts rotation depending on whether the player is grounded, creating a more natural movement feel.

It also handles head tilting by converting movement speed into an angle, which is smoothly interpolated for fluid transitions. This enhances the visual responsiveness of the character during acceleration and deceleration.

Personal Assessment

Overall, I think the project does what it was supposed to do. The main gameplay loop works, systems are connected, and things like inventory, saving, and player control feel stable in practice.

At the same time, I can see where things could be better. The structure is a bit tightly connected in some places, especially where systems directly reference each other. It works fine for a project of this size, but I can see how it could become harder to manage if the project grows.

If I continued working on it, I would try to make the architecture cleaner and more flexible. Probably by reducing direct dependencies between systems and using events or some kind of messaging approach instead. That would make everything easier to expand and maintain.